



Universidade do Minho
Escola de Engenharia
Licenciatura em Engenharia Informática

Unidade Curricular de Computação Gráfica

Ano Letivo de 2024/2025

Fase 4 - Coordenadas das normais e texturas

Grupo 5

Marco Soares Gonçalves (a104614) André Filipe Soares Pereira (a104275)
Salvador Duarte Magalhães Barreto(a104520) Leonardo Gomes Alves(a104093)

17 de Maio de 2025

CG

Fase 4 - Coordenadas das normais e texturas

Grupo 5

Marco Soares Gonçalves (a104614) André Filipe Soares Pereira (a104275)

Salvador Duarte Magalhães Barreto(a104520) Leonardo Gomes Alves(a104093)

17 de Maio de 2025

Índice

1	Introdução	1
2	Generator	2
2.1	Plano	3
2.1.1	Coordenadas da normal	3
2.1.2	Coordenadas de textura	3
2.2	Caixa	3
2.2.1	Coordenadas da normal	3
2.2.2	Coordenadas de textura	4
2.3	Cone	4
2.3.1	Coordenadas da normal	4
2.3.2	Coordenadas de textura	5
2.4	Esfera	6
2.4.1	Coordenadas da normal	6
2.4.2	Coordenadas de textura	6
2.5	Superfícies de Bézier	7
2.5.1	Coordenadas da normal	7
2.5.2	Coordenadas de textura	7
2.6	Anel	7
2.6.1	Coordenadas da normal	7
2.6.2	Coordenadas de textura	8
2.7	Novo formato dos ficheiros .3d	8
3	Engine	9
3.1	Alterações relativas à fase anterior	9
3.2	Texturas	10
3.3	Iluminação	11
4	Demo scene	12
5	Conclusão	13

Lista de Figuras

2.1	Função que desenhada a normal de cada vértice	2
2.2	Coordenadas de textura dos quatro vértices exteriores do plano	3
2.3	Cálculo das normais do cone	5
2.4	Base do cone	6
2.5	Fórmulas utilizadas para calcular as derivadas dos pontos	7
2.6	Novo formato dos ficheiros .3d	8
3.1	Novo estrutura Model	9
3.2	Novo estrutura Color	9
3.3	Nova estrutura Lights	10
3.4	Associação da textura ao <i>buffer</i>	11
4.1	Sistema solar com texturas e iluminação	12

1 Introdução

Este relatório documenta a última fase do desenvolvimento do projeto da disciplina de Computação Gráfica.

O objetivo desta fase é adicionar ao programa das fases anteriores as seguintes funcionalidades:

- Generator deve ser capaz de gerar coordenadas de texturas e normais para cada vértice;
- Engine deve ser capaz de renderizar iluminação e texturas.

De forma a suportar estas novas funcionalidades foi necessário fazer alterações no código, nomeadamente o código do parser XML, uma vez que é necessário fazer *parsing* de novas funcionalidades (iluminação, texturas).

Foi ainda atualizada a nossa demo scene, o sistema solar, de forma a suportar texturas para cada planeta e iluminação, no caso, por parte do Sol.

2 Generator

Uma vez que, é necessário inserir para cada vértice, as suas coordenadas de textura e a normal nesse ponto, foi necessário alterar o código de cada uma das figuras geométricas que o nosso *Generator* suporta.

Para tal, iremos abordar para cada uma, as alterações que foram efetuadas.

É importante realçar que, de forma a auxiliar a correta elaboração das normais de cada uma das figuras, implementamos uma funcionalidade que as desenha para cada vértice da figura. Esta funcionalidade foi realizada na função **draw** da *Engine*, correspondendo às últimas linhas que estão comentadas.

```
/* glDisable(GL_LIGHTING);

glColor3f(1.0f, 0.0f, 0.0f); // vermelho
glBegin(GL_LINES);
for (int j = 0; j < *n_vertices[i]; ++j) {
    Vertex v = listas[i]->get(j);

    float x = v.getX();
    float y = v.getY();
    float z = v.getZ();

    float nx = v.getnormalX();
    float ny = v.getnormalY();
    float nz = v.getnormalZ();

    glVertex3f(x, y, z);

    float scale = 0.2f;
    glVertex3f(x + nx * scale, y + ny * scale, z + nz * scale);
}

glEnd();
glEnable(GL_LIGHTING); */
```

Figura 2.1: Função que desenha a normal de cada vértice

2.1 Plano

2.1.1 Coordenadas da normal

Uma vez que o plano pertence ao plano XZ e, por isso, todos os pontos também pertencem a esse plano, é trivial que as normais dos pontos serão a "apontar para cima", no caso, as coordenadas da normal de todos os pontos será $(0,1,0)$.

2.1.2 Coordenadas de textura

Uma vez que a figura do Plano, desenhada no plano XZ é precisamente idêntica, em termos geométricos, às coordenadas de textura, cujas coordenadas **s** e **t** pertencem ao intervalo $[0, 1]$, basta associar a cada um dos vértices do plano, as suas coordenadas de textura.

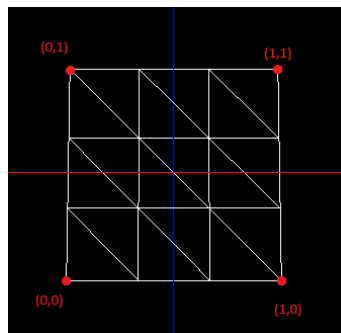


Figura 2.2: Coordenadas de textura dos quatro vértices exteriores do plano

Na imagem acima, temos definidos nos 4 vértices exteriores do plano, as suas respectivas coordenadas de textura. Desta forma, o cálculo das coordenadas para os restantes pontos será feita tendo em conta o número de divisões do plano. Estes cálculos têm apenas de assumir que o plano tem de largura 1.

Como exemplo, por mera observação, as coordenadas de textura dos pontos que definem a linha superior do plano serão as seguintes por ordem da esquerda para a direita: $(0,1)$; $(0.333,1)$; $(0.666,1)$; $(1,1)$.

2.2 Caixa

2.2.1 Coordenadas da normal

Para o cálculo das normais da caixa, o método será bastante parecido ao do plano, a única parte que temos que ter em conta é a orientação do plano que pretendemos saber a normal.

Por exemplo, o plano que define a parte superior da caixa, a sua normal será a "apontar para cima", logo será $(0,1,0)$. De forma similar, o plano que define a parte inferior da caixa, a sua normal será a "apontar para baixo" e, como tal, as coordenadas serão $(0,-1,0)$.

As restantes faces seguem a mesma lógica:

- A face frontal tem normal $(0,0,1)$;
- A face traseira tem normal $(0,0,-1)$;
- A face esquerda tem normal $(-1,0,0)$;
- A face direita tem normal $(1,0,0)$.

2.2.2 Coordenadas de textura

Este cálculo é bastante idêntico ao cálculo das coordenadas de textura do plano, uma vez que, a caixa é composta por planos. Assim, aplicamos o mesmo raciocínio sobre cada uma das seis faces, garantindo que a ordem dos vértices está correta.

2.3 Cone

2.3.1 Coordenadas da normal

No caso do cálculo das normais do cone, temos duas partes distintas a considerar: as normais base e as da superfície lateral.

A base é plana no plano XZ e, por isso, o cálculo da sua normal segue a mesma lógica dos planos. Logo, a normal dos pontos da base será claramente $(0,-1,0)$.

No entanto, a superfície lateral é inclinada e curva, exigindo uma abordagem diferente. É possível verificar que normal varia consoante a posição angular em que nos encontramos, ou seja, ao longo da circunferência porém, a inclinação é sempre a mesma. Neste caso, a componente do vetor que será sempre constante, será a componente Y. Como vamos calcular a normal em ordem aos valores de α , a nossa componente Y poderá ser $\frac{\text{radius}}{\text{height}}$, sendo radius o raio da base do cone e height a altura do cone.

Desta forma, basta calcular as componentes de X e Z, que serão calculadas através dos valores de α e, é trivial que a componente de X será $\cos \alpha$ e Z será $\sin \alpha$.

De forma a facilitar a visualização do cálculo da normal, desenhámos a imagem abaixo que representa o cálculo da normal para um certo ângulo α .

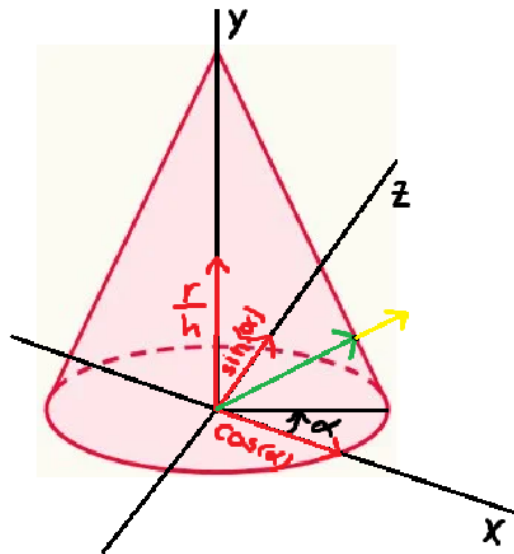


Figura 2.3: Cálculo das normais do cone

É importante lembrar que a nossa normal necessita de ser normalizada e, esta está representada a amarelo na imagem acima.

2.3.2 Coordenadas de textura

No caso do cone, a superfície lateral é desenhada como um conjunto de quadriláteros, resultantes da divisão do cone em slices e stacks.

Para conseguirmos mapear corretamente a textura, é necessário associar a cada vértice da superfície lateral do cone uma coordenada de textura (s , t), onde s irá corresponder à posição angular, ou seja da *slice*, e t à posição vertical, ou seja da *stack*.

O valor de s é portanto calculado como a razão entre o índice da slice atual e o número total de slices, ou seja, $s = \frac{j}{\text{slices}}$.

O valor de t é calculado como $1 - \frac{i}{\text{stacks}}$, onde i representa a stack atual. É importante mencionar que o número 1 é necessário neste cálculo uma vez que sem ele a textura iria ficar mapeada ao contrário, verticalmente.

No caso da base do cone, as coordenadas de textura são calculadas de forma semelhante a um círculo. Uma vez que o centro da textura é no ponto com coordenadas (0.5, 0.5), basta usar o ângulo alpha juntamente com **sin** e **cos** que conseguimos determinar a posição de cada ponto na textura.

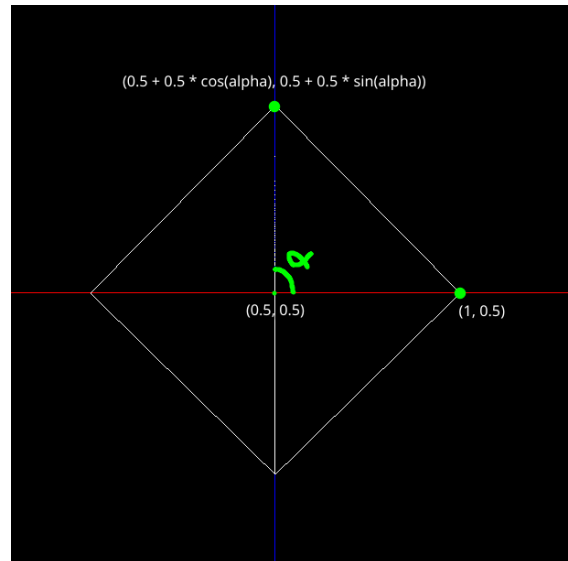


Figura 2.4: Base do cone

2.4 Esfera

2.4.1 Coordenadas da normal

O cálculo das normais da esfera é bastante simples, uma vez que a normal de cada ponto, é precisamente um vetor da origem da esfera até esse ponto. Apenas temos que ter o cuidado de normalizar o vetor de forma a ficar com a norma igual a 1.

Portanto, para um certo ponto $p(x,y,z)$ pertencente à superfície da esfera, a sua normal será $(x_normalizado, y_normalizado, z_normalizado)$.

2.4.2 Coordenadas de textura

Para calcular as coordenadas de textura é bastante simples, uma vez que temos que percorrer os valores de α , para o caso da componente s , obtendo assim os pontos ao longo das slices (longitude) e, de forma semelhante, percorrer os valores de β para a componente t , que corresponde às stacks (latitude).

Assim, a componente s é calculada através de $\frac{\alpha}{2\pi}$. Desta forma, à medida que percorremos todas as slices (de 0 a 2π), a coordenada s varia de 0 a 1, cobrindo toda a largura da textura.

A componente t é calculada através de $1 - \frac{\beta + \frac{\pi}{2}}{\pi}$. É importante mencionar que é necessário ter o valor de 1 uma vez que, sem ele, a textura ficaria ao contrário (verticalmente).

2.5 Superfícies de Bézier

2.5.1 Coordenadas da normal

Para calcular as normais de uma superfície de Bézier, basta utilizar as derivadas parciais de cada um dos pontos da superfície. Desta forma, obtém-se dois vetores (v , u), que são vetores tangentes à superfície nesse ponto.

Desta forma, com os vetores calculados, basta fazer o *cross-product* e obtemos a normal desse ponto.

$$\frac{\partial p(u,v)}{\partial u} = [3u^2 \quad 2u \quad 1 \quad 0] M \begin{bmatrix} P_{00} & P_{01} & P_{02} & P_{03} \\ P_{10} & P_{11} & P_{12} & P_{13} \\ P_{20} & P_{21} & P_{22} & P_{23} \\ P_{30} & P_{31} & P_{32} & P_{33} \end{bmatrix} M^T V^T$$
$$\frac{\partial p(u,v)}{\partial v} = U M \begin{bmatrix} P_{00} & P_{01} & P_{02} & P_{03} \\ P_{10} & P_{11} & P_{12} & P_{13} \\ P_{20} & P_{21} & P_{22} & P_{23} \\ P_{30} & P_{31} & P_{32} & P_{33} \end{bmatrix} M^T \begin{bmatrix} 3v^2 \\ 2v \\ 1 \\ 0 \end{bmatrix}$$

Figura 2.5: Fórmulas utilizadas para calcular as derivadas dos pontos

2.5.2 Coordenadas de textura

As coordenadas de textura das superfícies de Bézier são atribuídas diretamente a partir dos parâmetros u e v utilizados na função de processar cada patch (**processPatch**). Como u e v variam entre 0 e 1, os pontos já estão normalizados nesse intervalo e respeitam a tesselação, desta forma, podem ser imediatamente aplicados nas coordenadas de textura.

2.6 Anel

2.6.1 Coordenadas da normal

As coordenadas da normal dos vértices é simples de calcular uma vez que o anel é composto por duas faces no plano XZ, uma a apontar para "cima" (eixo positivo do Y) e outra para "baixo" (eixo negativo do Y). Desta forma, claramente que o vetor da normal dos vértices dos plano será (0,1,0) e (0,-1,0), respectivamente.

2.6.2 Coordenadas de textura

Para o anel, o mapeamento da textura foi feito de forma a que a imagem da textura seja "esticada" à volta do anel, cobrindo toda a superfície entre o raio interior e o exterior.

Assim, a coordenada s corresponde à posição angular ao longo do anel. Ou seja é calculada da seguinte forma: $\frac{\alpha}{2\pi}$. Assim, à medida que percorremos o anel, a coordenada s varia de 0 a 1, cobrindo toda a largura da textura.

Quanto à coordenada t , apenas pode ter 2 valores, 1 caso se tratar de um ponto pertencente ao raio exterior e 0 ao um ponto do raio interior.

2.7 Novo formato dos ficheiros .3d

Uma vez que foi necessário acrescentar novas coordenadas para cada um dos vértices de uma certo objeto, foi necessário atualizar o formato dos ficheiros .3d, que são processados pela *Engine*, para o seguinte formato:

```
324
-1.000000,-1.000000,-1.000000; 0.000000,-1.000000,0.000000; 0.000000,1.000000
-0.333333,-1.000000,-1.000000; 0.000000,-1.000000,0.000000; 0.333333,1.000000
-1.000000,-1.000000,-0.333333; 0.000000,-1.000000,0.000000; 0.000000,0.666667
-0.333333,-1.000000,-0.333333; 0.000000,-1.000000,0.000000; 0.333333,0.666667
-1.000000,-1.000000,-0.333333; 0.000000,-1.000000,0.000000; 0.000000,0.666667
-0.333333,-1.000000,-1.000000; 0.000000,-1.000000,0.000000; 0.333333,1.000000
```

Figura 2.6: Novo formato dos ficheiros .3d

Cada linha representa um vértice da figura geométrica onde, as primeiras três coordenadas representam as coordenadas do vértice no espaço global, as próximas três coordenadas representam a normal desse ponto e, por fim, as últimas duas coordenadas da linha representam as coordenadas de textura.

3 Engine

3.1 Alterações relativas à fase anterior

Como falado na introdução do trabalho, foi necessário fazer alterações ao *parser* dos ficheiros XML e, para tal, fizemos algumas alterações à estrutura do **World**.

De forma a suportar as texturas, foi necessário adicionar à nossa struct **Model**, uma string que dita o nome da nossa textura e ainda, uma estrutura **Cor**, necessária para no caso da textura não ser especificada, ser essa a cor utilizada por parte do objeto.

```
struct Model {  
    string file;  
    string texture_file;  
    Color color;  
};
```

Figura 3.1: Novo estrutura Model

Assim, a estrutura da cor, **Color**, tem as seguintes variáveis que, por sua vez, têm strings como os atributos correspondentes aos valores de R, G, B e *value* (no caso da **Shininess**).

```
struct Color {  
    Diffuse diffuse;  
    Ambient ambient;  
    Specular specular;  
    Emissive emissive;  
    Shininess shininess;  
};
```

Figura 3.2: Novo estrutura Color

Por fim, para ser capaz de renderizar corretamente as luzes, também foi inserida a estrutura **Lights** na estrutura **World**, que é um vetor com as luzes que foram *parsadas* do XML. Cada uma dessas luzes tem diferentes atributos e três tipos diferentes, *spotlight*, *directional* ou *point*.

```
struct Light {
    string type;
    float posX, posY, posZ;
    float dirX, dirY, dirZ;
    float cutoff;
};

struct Lights {
    vector<Light> lights;
};

struct World {
    Window window;
    Camera camera;
    Group group;
    Lights lights;
};
```

Figura 3.3: Nova estrutura Lights

3.2 Texturas

Já com as alterações feitas no *Generator* e no *parser* XML, falta fazer o *load* correto das texturas.

Para tal, temos a função *loadTexture* que, para cada um dos objetos, caso este tenha uma textura a si associada, faz o *load* da mesma para o programa usando o **DevIL**. Assim, existe um mapa na *engine* que guarda todas as texturas carregadas, sendo a chave de cada elemento o nome da textura.

Por fim, na nossa função de colocar as figuras/objetos geométricos na *engine*, é feita a associação da dada textura ao seu buffer correspondente. Da mesma forma, a função *draw* associa corretamente a cada figura o seu dado buffer através do array **textures**.

```
glGenBuffers(1, &textures[indice_list]);  
glBindBuffer(GL_ARRAY_BUFFER, textures[indice_list]);  
glBufferData(GL_ARRAY_BUFFER, sizeof(float) * vertexCount * 2, t, GL_STATIC_DRAW);
```

Figura 3.4: Associação da textura ao *buffer*

3.3 Iluminação

Referente à iluminação, o processo foi bastante parecido com o das texturas, uma vez que é necessário também modificar as funções ***draw*** e ***prepareFigures*** da mesma forma.

Foi criada a função `setupLights` que trata de colocar os vários tipos de luzes nas suas posições e modos devidos, utilizando as funções ***glLightfv*** e ***glLightf*** com os parâmetros necessários.

De seguida, serão apresentados os três tipos de luzes diferentes e os valores que têm a si associados:

- Point - É apenas necessário indicar um vetor que defina a posição que a luz está (posX, posY, posZ);
- Directional - É necessário indicar um vetor que defina a direção da luz (dirX, dirY, dirZ);
- Spotlight - É necessário indicar um vetor que defina a sua posição e outro para a direção de luz. É ainda requerido um valor de *cutoff*.

Referente à iluminação dos objetos, estes podem ter quatro componentes de cor: difusa, ambiente, especular e emissiva. Este valores provém exatamente do *parsing* da textura, nomeadamente da variável *color*, uma vez que estas são necessários para a iluminação correta da figura geométrica.

4 Demo scene

Relativamente à demo scene, foi nos pedido para, relativamente ao sistema solar da fase anterior, inserir luzes e texturas nos planetas.

No que diz respeito às texturas, recorremos aos *website's* fornecidos pelos docentes e, fizemos o correto mapeamento em cada um dos planetas do sistema solar.

Quanto à iluminação, decidimos criar uma luz do tipo **point** no lugar do Sol e, desta forma, ilumina corretamente o sistema solar e todos os planetas. É importante mencionar também que, para que o sol fique com a correta iluminação (ou seja, brilhante), é necessário definir a sua cor emissiva como branca ($RGB = (255,255,255)$).

Para além disso, adicionámos ainda o movimento de rotação dos planetas por eles próprios, ou seja, cada planeta não só realiza o movimento de rotação em torno do Sol, mas também executa uma rotação sobre o seu próprio eixo.

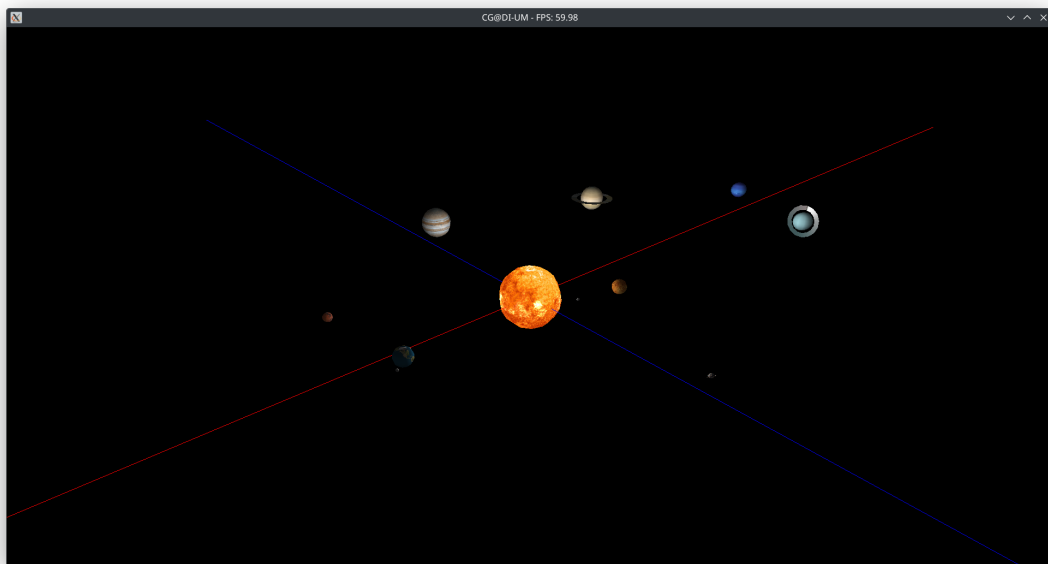


Figura 4.1: Sistema solar com texturas e iluminação

5 Conclusão

Nesta quarta e última fase do projeto, implementámos as normais e coordenadas de textura dos objetos, o sistema de iluminação e o suporte para texturas por parte da Engine. Estes avanços permitiram-nos dar um passo significativo na aproximação entre os nossos modelos e uma representação visual mais realista, baseada nos conceitos que foram lecionadas nesta UC.

Achámos que foi um projeto bastante interessante e diferente dos que nos foram propostos durante o curso e, desta forma, desenvolvemos conhecimento básico na área de computação gráfica.

Por fim, concluímos este trabalho reconhecendo que tivemos um bom aproveitamento, uma vez que implementámos todas as funcionalidades que nos foram requeridas, cumprindo integralmente todos os testes.